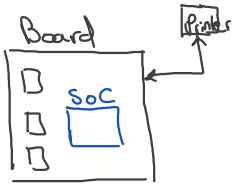
Kernel

Saturday, September 13, 2025 9:34 AM

① Repo structure

Vendor (company self board)



Objective

↳ Kernel

↳ print "hello"

Shutdown

Kernel

↳ intel 387

↳ X86

↳ arm

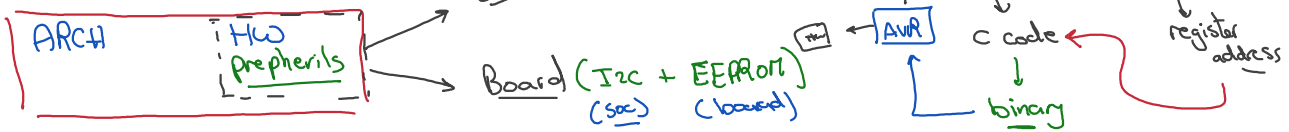
↳ Risc-v

↳ execute code

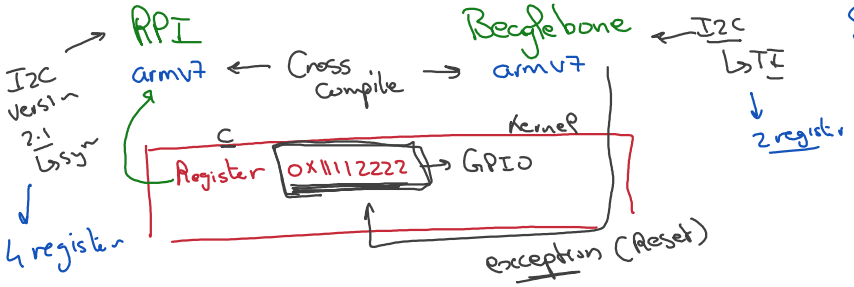
@ owner that interact with the HW

ⓑ give the user interface to act upon it (system call)

ⓒ Respond on any HW change



Baremetal



Solution ①

↳ remove addresses from Source Code

↳ & replace it with variable

c → represent Driver I2C

Synopsis

TI

I will not access any register

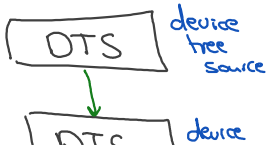
Boards (armv7)

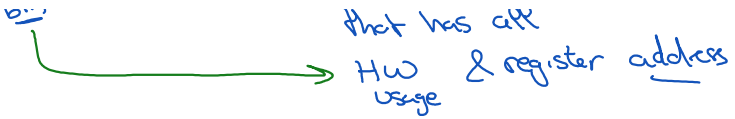
with same architecture

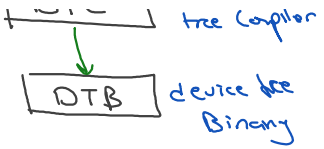
Kernel

DTB

Configuration





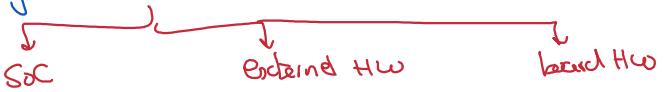


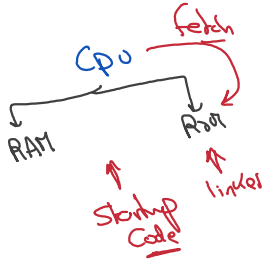
① when Kernel is activated
it will retrieve the addresses
from DTB

```
struct adp5520_gpio_platform_data *pdata = dev_get_platdata(&pdev->dev);
```

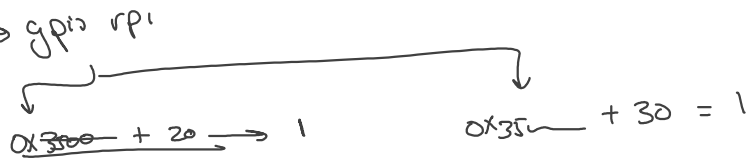
← the line that retrieve the HW data from DTB

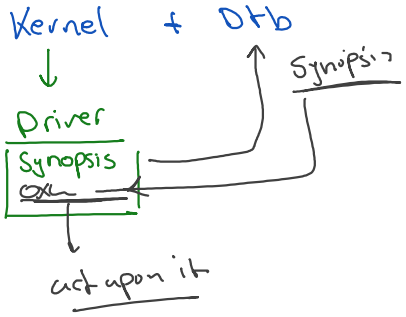
② DTB
↳ Configuration file which describe your board

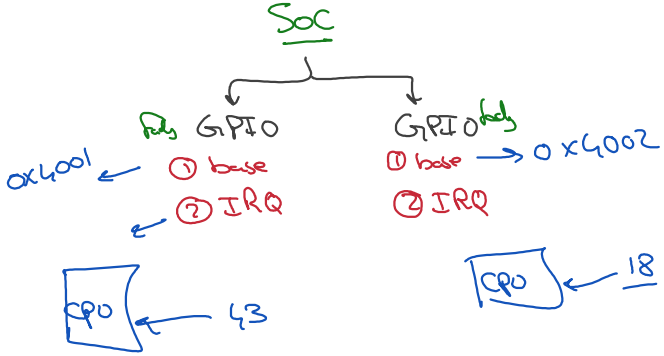




③ Driver
↳ gpio rpi







KBuild

↳ Kernel build

↳ Compile the Kernel Depends on the arch (--no-stdlib -freestanding)

Image ← Implementation

↳ Baremetal

Kernel headers ↑ use headers provided by the Kernel

Std library C

During writing any Linux module

↳ you must use the Linux header file (following version)

V.3 proc.h

write_proc(- - - ->

V.4 proc.h ↔ Source code Ker- V.4

(K) write_proc( )

Linux ← fork → Board ← Drivers

rpi

↳ locally